Day 1

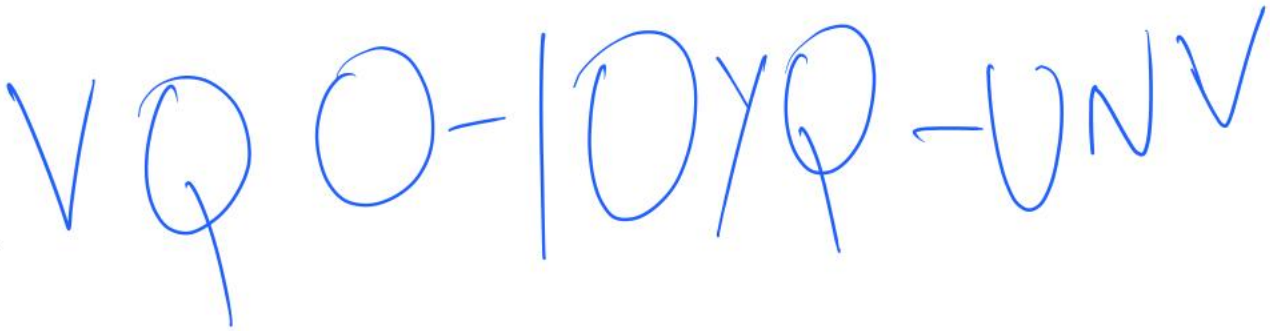
Thursday, 13 August 2026 9:13 AM

n[4]

4	4	4	4	4	4	4
4	3	3	3	3	3	4
4	3	2	2	2	3	4
4	3	2	1	2	3	4
4	3	2	2	2	3	4
4	3	3	3	3	3	4
4	4	4	4	4	4	4

n[3]

3	3	3	3	3
3	2	2	2	3
3	2	1	2	3
3	2	2	2	3
3	3	3	3	3



	0	1	2	3	4	5	6
0	4	4	4	4	4	4	4
1	4	3	3	3	3	3	4
2	4	3	2	2	2	3	4
3	4	3	2	1	2	3	4
4	4	3	2	2	2	3	4
5	4	3	3	3	3	3	4
6							

n[4] size[7]

i[3] j[3]

top → i left → j

down → size-1-i right → size-1-j

top[3] left[3]

198. House Robber

Solved

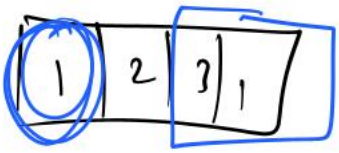
Medium | Topics | Companies

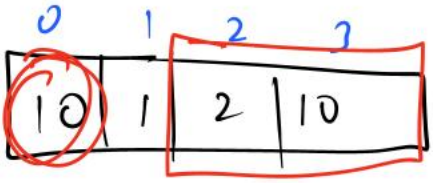
You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

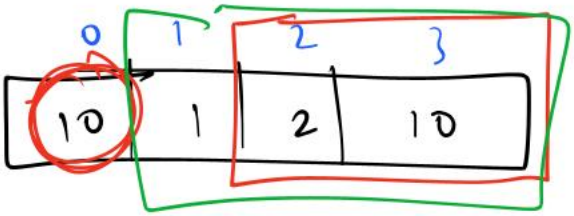
Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight* ***without alerting the police***.

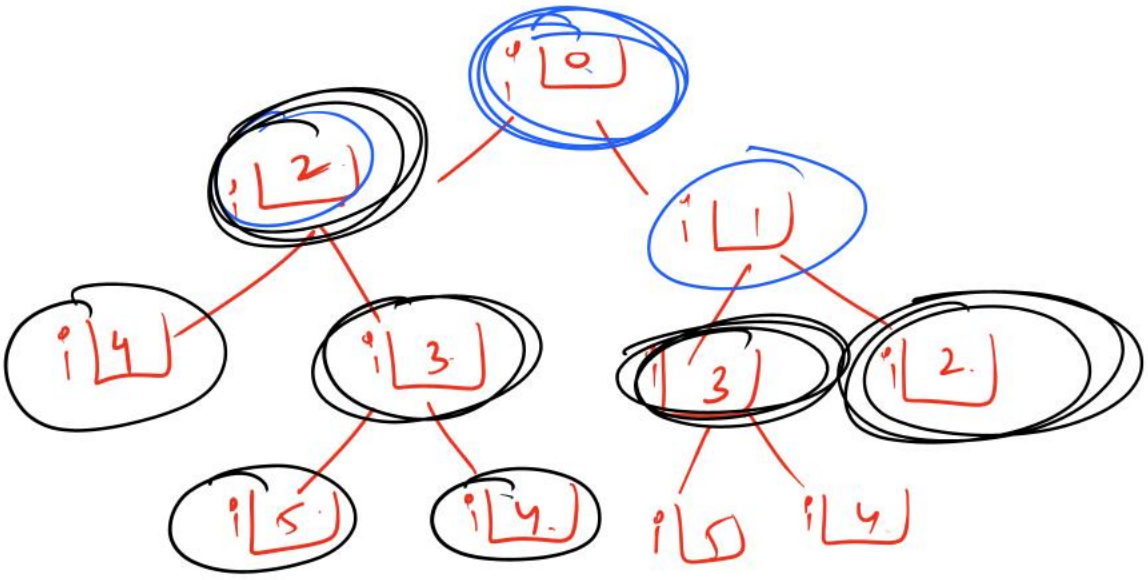
Example 1:

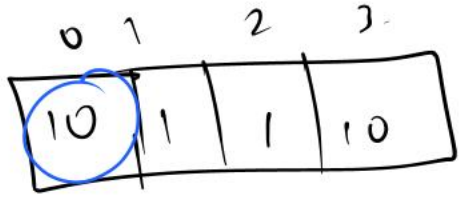
```
Input: nums = [1,2,3,1]
Output: 4
```

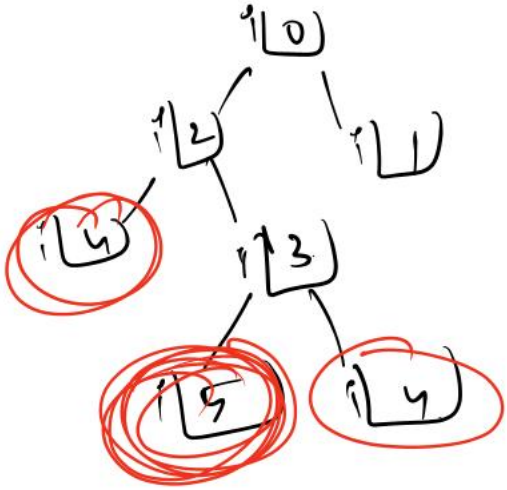






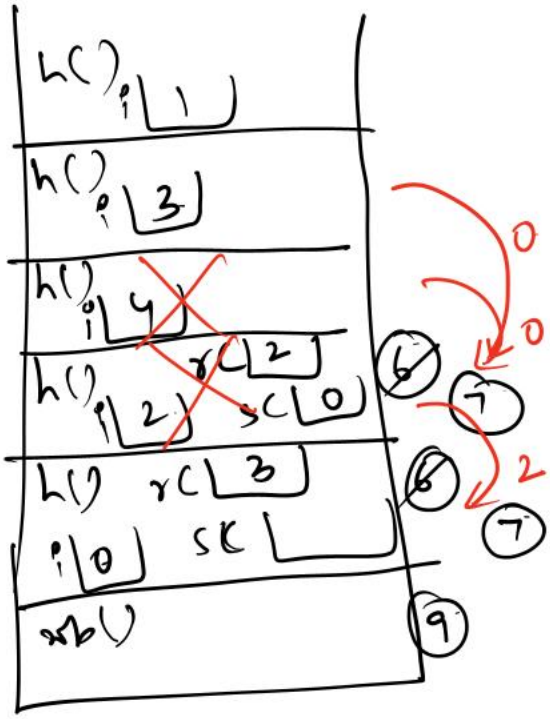


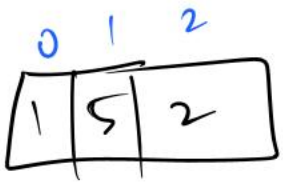


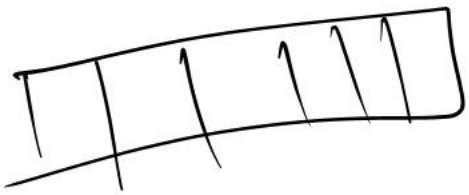


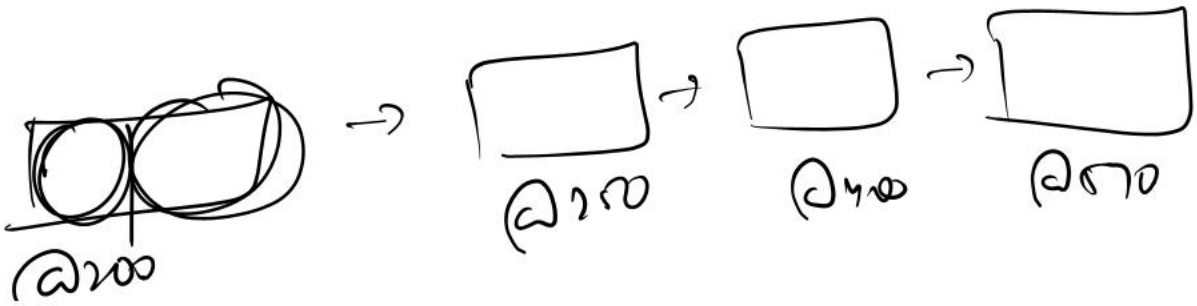
4

```python
class Solution:
    def rob(self, nums: List[int]) -> int:
        def helper(i) :
            if i >= len(nums) :
                return 0
            robCurrent = nums[i] + helper(i + 2)
            skipCurrent = helper(i + 1)
            return max(robCurrent, skipCurrent)
        return helper(0)
```

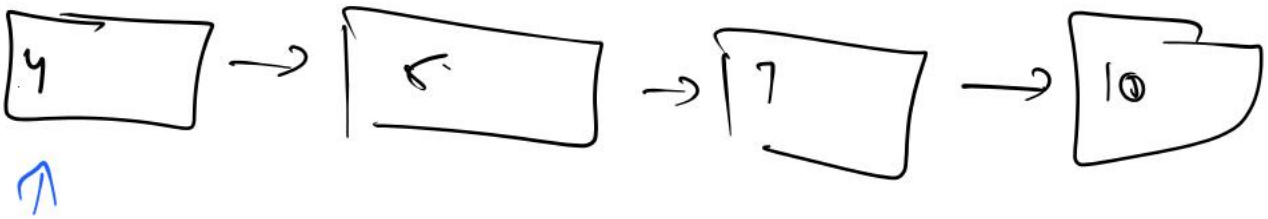




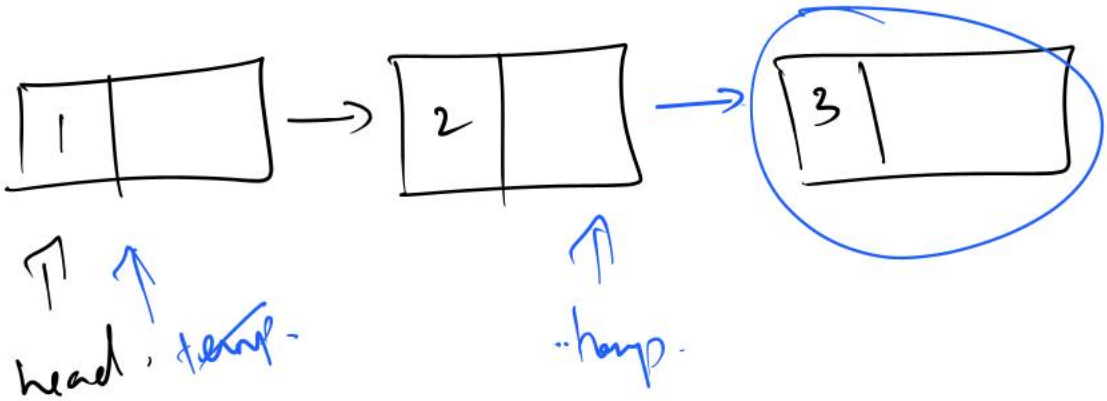




9 5 7 10 -1



1 2 3




```python
def buildList() :
    data = int(input('Enter head data : '))
    if data == -1 :
        return None
    head = Node(data)
    temp = head
    while True :
        data = int(input('Enter node data : '))
        if data == -1 :
            break
        newNode = Node(data)    # Create a newNode
        temp.next = newNode # Connect current Node with previous
        temp = newNode
    return head
```

Linked List End Insertion

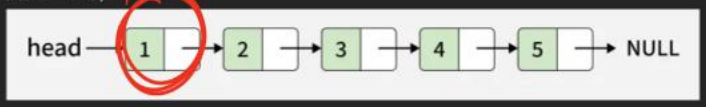
Solved

Difficulty: **Basic** Accuracy: **43.96%** Submissions: **417K+** Points: **1** Average Time: **20m**

You are given the **head** of a Singly Linked List and a value **x**, insert that value **x** at the end of the LinkedList and return the **head** of the modified Linked List.

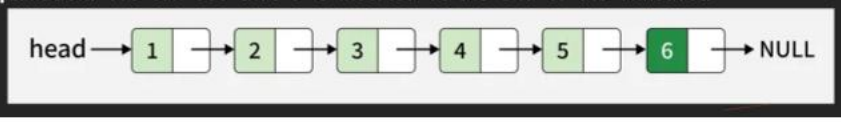
Examples :

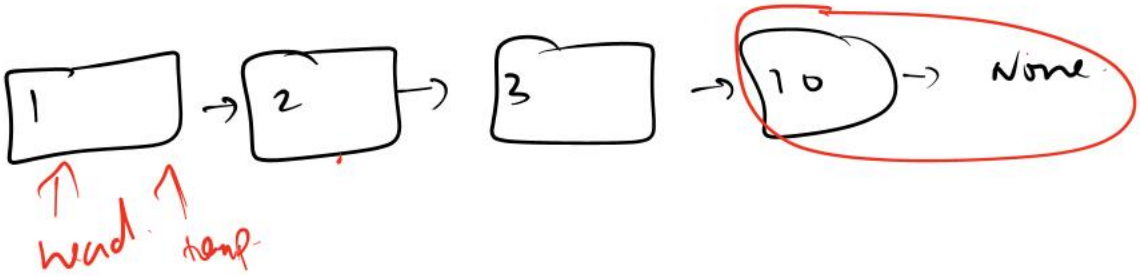
Input: x = 6,



Output: 1 -> 2 -> 3 -> 4 -> 5 -> 6

Explanation: We can see that 6 is inserted at the end of the linkedlist.






```python
class Solution:
    def insertAtEnd(self, head, x):
        newNode = Node(x)
        if head is None :
            return newNode
        temp = head
        while temp.next is not None :
            temp = temp.next
        temp.next = newNode
```

] → If linked list is empty

→ T.C → O(n)

S.C → O(1)

```
        return head
```

Linked List Delete at Position

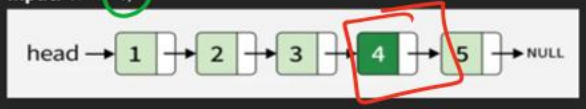
Difficulty: **Easy** Accuracy: **39.85%** Submissions: **259K+** Points: **2**

Given the **head** of a linked list and an integer **x**, delete the node at position **x** and return the updated head of the linked list.

Note: Positions use 1-based indexing.

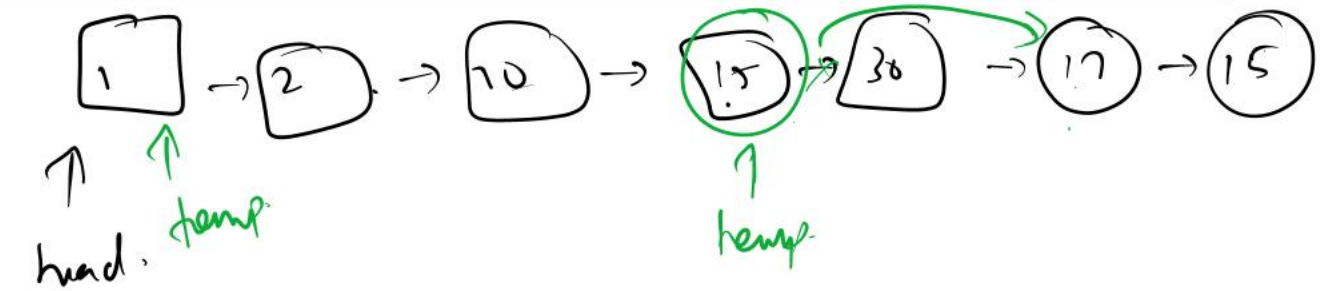
Examples:

Input: x = 4,



Output: 1 -> 2 -> 3 -> 5

Explanation: After deleting the node at the 4th position, the linked list is as



temp.next = temp.next.next

x | 5

```
class Solution:
    def deleteNode(self, head, x):
        #code here
        if x == 1 :
            return head.next
        temp = head
```

→ T.C → O(n)

→ S.C → O(1)

```
        for i in range(x - 2) :
            temp = temp.next
        temp.next = temp.next.next
        return head
```

876. Middle of the Linked List

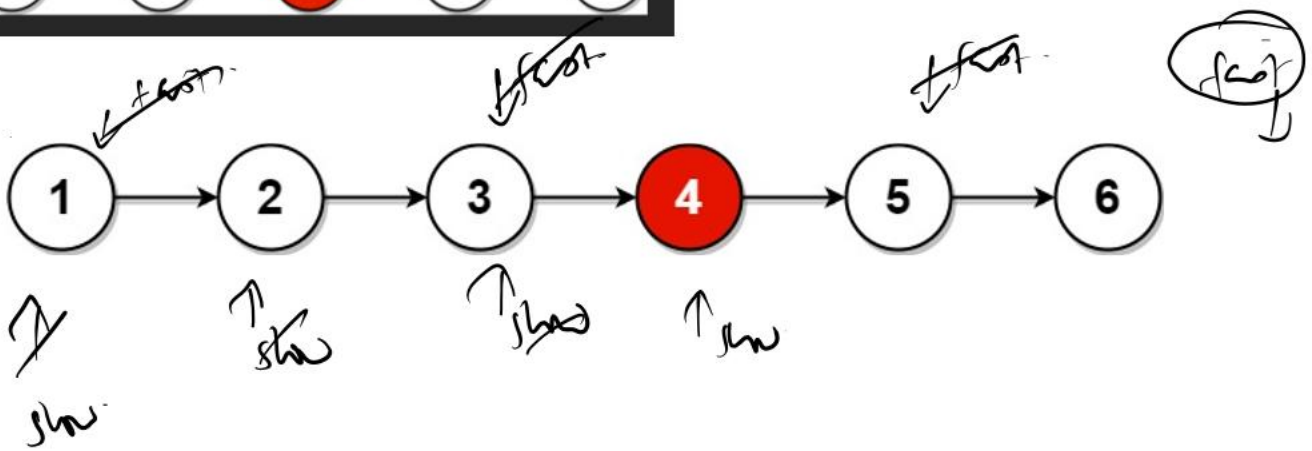
Solved

Easy | Topics | Companies

Given the `head` of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:



slow = slow.next

fast = fast.next.next

```python
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow, fast = head, head
        while fast is not None and fast.next is not None :
            slow = slow.next
            fast = fast.next.next
        return slow
```

↓ ↓ ↓

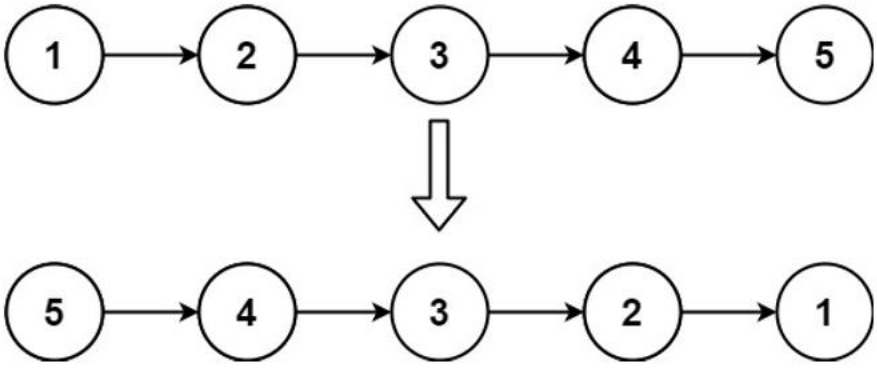
P.NO: 876 T.C → O(n) S.C → O(1)

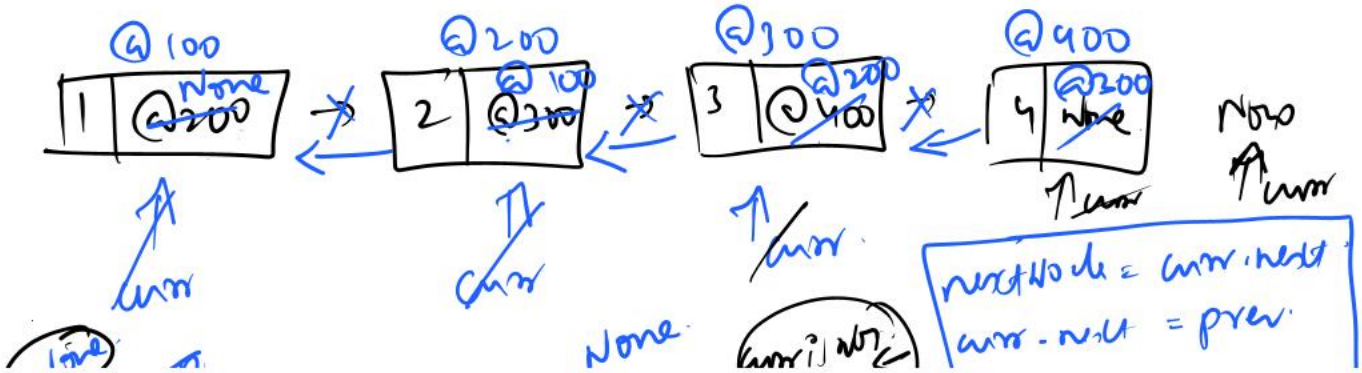
206. Reverse Linked List

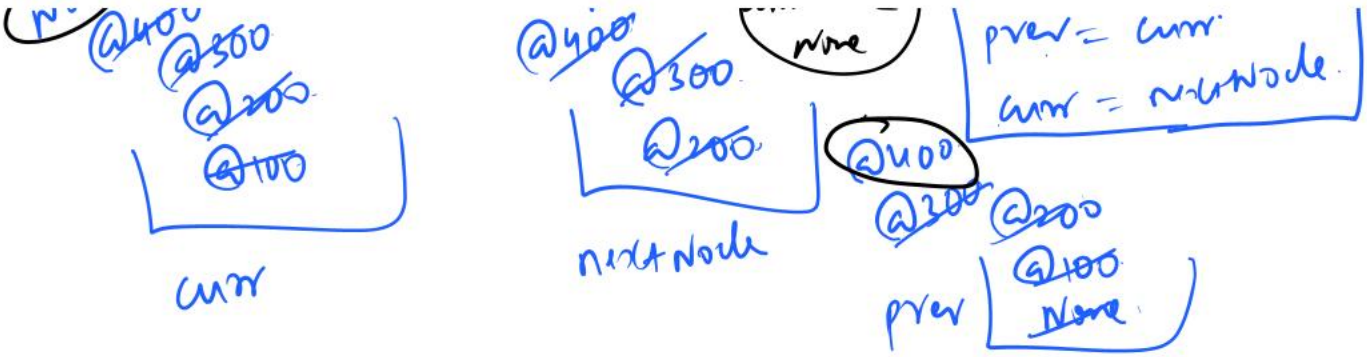
Solved

Easy | Topics | Companies

Given the `head` of a singly linked list, reverse the list, and return *the reversed list*.





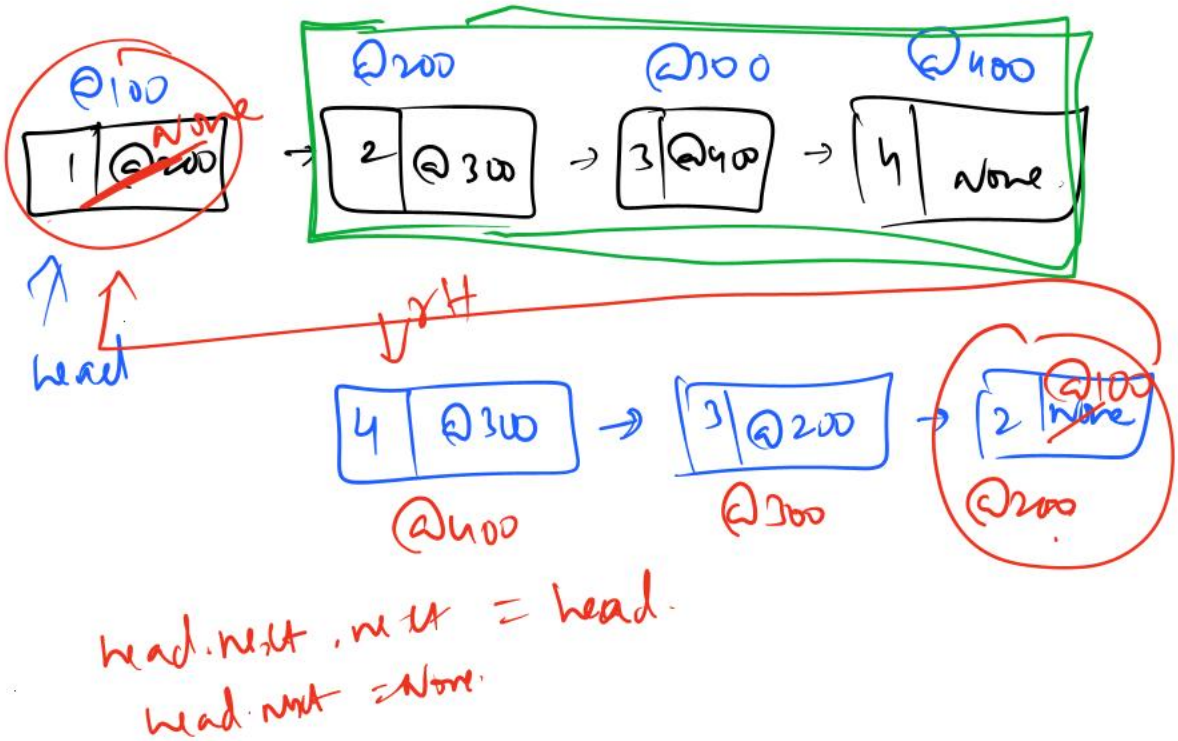



```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev, curr = None, head
        while curr is not None :
            nextNode = curr.next
            curr.next = prev
            prev = curr
            curr = nextNode
        return prev
```

→ P.No: 206

→ T.C → O(n)

→ S.C → O(1)



head.next.next = head

head.next = None

```python
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if head is None or head.next is None :
```

```
        return head
    reversedHead = self.reverseList(head.next)
    head.next.next = head
    head.next = None
    return reversedHead
```

T.C → O(n).

S.C → O(n)

Call stack space

141. Linked List Cycle

Solved

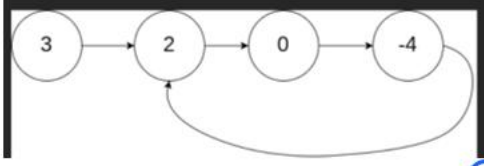
Easy | Topics | Companies

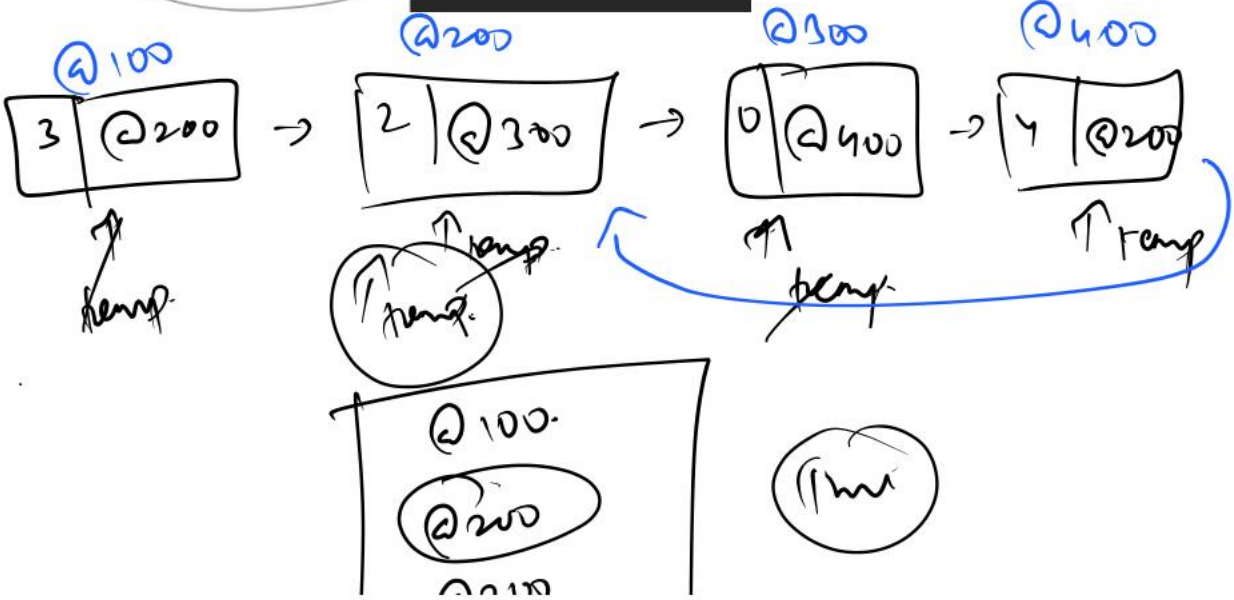
Given `head`, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to. **Note that `pos` is not passed as a parameter.**

Return `true` *if there is a cycle in the linked list*. Otherwise, return `false`.

Example 1:





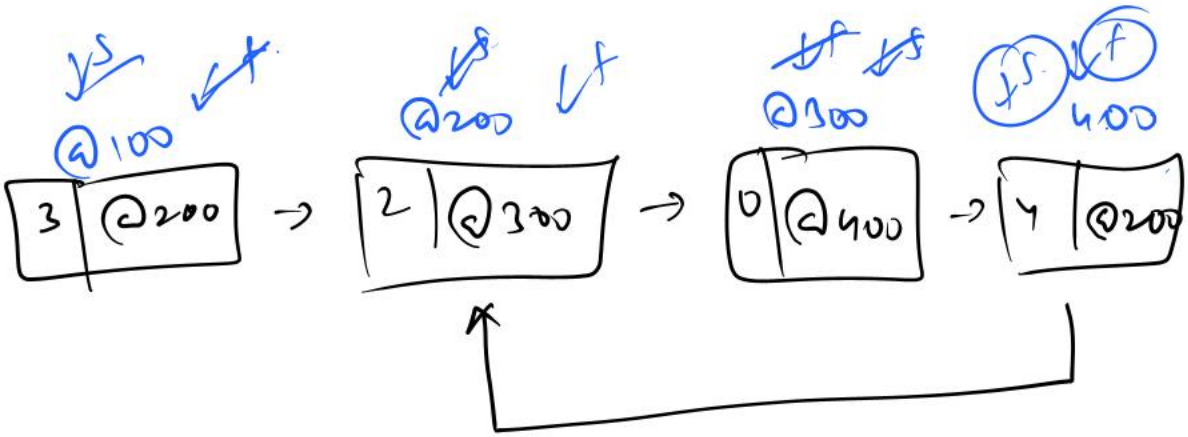

```python
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        seen = set()
        temp = head
        while temp is not None :
            if temp in seen :
                return True
            seen.add(temp)
            temp = temp.next
        return False
```

T.C → O(n)

S.C → O(n)

↓

set's space.




```python
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow, fast = head, head
        while fast and fast.next :
            slow = slow.next
            fast = fast.next.next
            if slow == fast :
                return True
        return False
```

83

21

2